

2026 年 5 月 13 日

Arduino ～計画書と設計書～ オーケストラ

チーム 23

25G1065 : 塩澤 匠生

目次

第 1 章	システムの基本設計	1
1.1	機能一覧	1
1.1.1	要求一覧	1
1.1.2	機能分解 (FBS)	2
1.2	システム構成図	3
1.2.1	全体ブロック図	3
1.2.2	ノード役割分担	3
1.2.3	データフロー	4
1.2.4	採用アーキテクチャ (EMA) の方針	4
1.3	操作インターフェース設計	5
1.3.1	ユーザー操作	5
1.3.2	LED 表示仕様	5
1.3.3	通信プロトコル方針 (CTRL / BEAT / NOTE)	6
1.3.4	通信方式 (WiFi UDP + USB Serial の二系統)	7
1.3.5	パケロス・ジッタ対策	7
1.4	状態遷移図	8
1.4.1	指揮者ノード (node_01)	8
1.4.2	楽器ノード (node_02~05)	9
第 2 章	システムの詳細設計	11
2.1	部品一覧	11
2.2	ハードウェア設計	13
2.2.1	配線・ピン	13
2.2.2	ネットワーク構成	14
2.3	ソフトウェア設計	15
2.3.1	ファイル構成	15
2.3.2	オブジェクト設計 (クラス図)	16
2.3.3	関数設計	17
2.4	処理フローの設計	26
2.4.1	共通: 3 フェーズ実行モデル	26
2.4.2	指揮者ノード (node_01)	26
2.4.3	楽器ノード (node_02~05)	33

2.5	テストの設計	39
2.5.1	設計目標値	39
2.5.2	検証の方向性	40
2.5.3	実機検証で確定する仮値・未確定事項	41
第 3 章	生成 AI の利用について	43

第 1 章 システムの基本設計

本章では、「Arduino オーケストラ」（指揮者ノード 1 + 楽器ノード 4 の計 5 ノード）のうち塩澤担当範囲（指揮者ノード node_01 と楽器ノード node_02～node_05 のファームウェア）を対象に、システム全体の機能・構成・操作インターフェース・状態遷移を基本設計レベルでまとめる。楽器構成は **金管 3 + ドラム 1** とし、各楽器ノードは**専用の Mac (Processing 実行) と 1 対 1 で USB 接続**される。Mac 側の音色合成・スピーカ出力（梅澤担当）、および楽譜を作る楽譜エディタ（齋藤担当）は本書のスコープ外で、それぞれデータ受け渡し境界（NOTE パケット、楽譜データ形式）のみを定義する。通信は **ノード間（指揮者 → 楽器） = WiFi UDP マルチキャスト**、**Arduino-Mac 間（楽器 → 各 Mac） = USB Serial (UART, Serial.write)** の二系統に分離した構成を採る。

ファームウェアの設計原則は、外部 OSS であり著者本人が公開している **Embedded-Module-Architecture (以下 EMA)**¹⁾ に全面準拠する。EMA の中核ルール（IModule インターフェース、3 フェーズループ、SystemData と ProjectConfig の集約）は §1.2 で位置づけを示し、第 2 章で具体化する。

1.1 機能一覧

本節ではファームウェアが満たすべき要求と、それを機能分解構造（FBS）で整理した機能一覧を示す。本書スコープは Arduino 系のファームウェアのみであり、楽器音色や発表準備などチーム全体側の要素は含めない。

1.1.1 要求一覧

本ファームウェアが満たすべき要求を表 1.1 に示す。「(S)」はストレッチ要求（必達機能の完成後に着手する）であり、R-Internal は外部から見えにくい設計上必要な内部要求である。

表 1.1: 要求一覧

ID	要求	内容
R-1	5 台同期演奏	指揮者 1 + 楽器 4 の計 5 ノードが、指揮者の拍に合わせて同期して演奏する

1) <https://github.com/takushio2525/Embedded-Module-Architecture>

ID	要求	内容
R-2	輪楽曲の再生	主旋律 3 パート（金管）＋リズム 1 パート（ドラム）以上で構成される輪楽曲を演奏できる。各パートは入り拍をずらして輪唱になる
R-3	可変テンポ追従	指揮者の振りの速さの変化に楽器側のテンポが追従する
R-4 (S)	強弱制御	指揮者の振りの大きさに応じて発音の強弱（velocity）が変わる
R-Internal-1	通信の信頼性	数 % 程度のパケロスが起きても演奏が破綻しない
R-Internal-2	CPU リソース	各ノードが 200 Hz の IMU サンプリングまたは UDP 受信を取りこぼさずに回せる範囲の処理量に収める
R-Internal-3	起動時間	電源投入から演奏可能になるまでが、実演のセットアップ時間として許容できる範囲（数秒）に収まる

これらの要求に対する数値目標（設計目標値）は §2.5.1 に、検証の方向性は §2.5 にまとめる。

1.1.2 機能分解（FBS）

F1. 指揮情報抽出（node_01） F1.1 IMU 6 軸の取得, F1.2 信号前処理（IIR LPF + ノルム）, F1.3 拍検出（振り下ろしピーク）, F1.4 テンポ推定（拍間隔→BPM）, F1.5 強弱推定（振幅→velocity, ストレッチ）, F1.6 演奏状態管理.

F2. 指揮情報配信（node_01） F2.1 CTRL 送出（BPM・velocity・state）, F2.2 BEAT 送出（拍トリガ）, F2.3 SoftAP 起動・維持（自身がアクセスポイントを兼ね、楽器ノードを収容）.

F3. 楽譜進行（node_02～05） F3.1 CTRL/BEAT 受信, F3.2 楽譜データ保持（C 配列・パート別）, F3.3 BEAT 同期での音符進行, F3.4 NOTE イベント生成, F3.5 パート固有ロジック（PART_ID・startBeatNo）.

F4. 発音情報配信（node_02～05 → PC） F4.1 NOTE パケット生成, F4.2 Processing への USB Serial 送信（Serial.write(), CDC 1:1 接続）.

F5. 共通基盤（全ノード） F5.1 IModule 登録・3 フェーズループ, F5.2 ModuleTimer による周期実行, F5.3 SystemData/ProjectConfig 集約, F5.4 起動シーケンス・診断 LED, F5.5 init 失敗リトライ・パケロス時のフォールバック.

ストレッチ機能（F1.5 強弱推定）は必達機能の実装完了後に着手する. 詳細設計ではインターフェースだけ用意し, 初期実装ではスタブで通す構成にする.

1.2 システム構成図

1.2.1 全体ブロック図

5 ノードと Mac × 4 の関係を図 1.1 に示す. 指揮者 **node_01** 自身が **SoftAP（ESP32-S3 のソフトウェア AP）** を立て, 楽器 **node_02~05** は STA としてこれに接続する. 外部ルータやスマホテザリングは不要. **node_01** は CTRL/BEAT を **UDP マルチキャスト** で送信し, **node_02~05** はそれぞれグループに join して受信する. 各楽器は楽譜を進行させながら **NOTE** を **USB Serial（Serial.write）** で**専用 Mac（1 対 1 接続）** へ送信する. Arduino-Mac 間は WiFi を介さず, 給電と Serial 通信を兼ねた USB 接続で直結する.

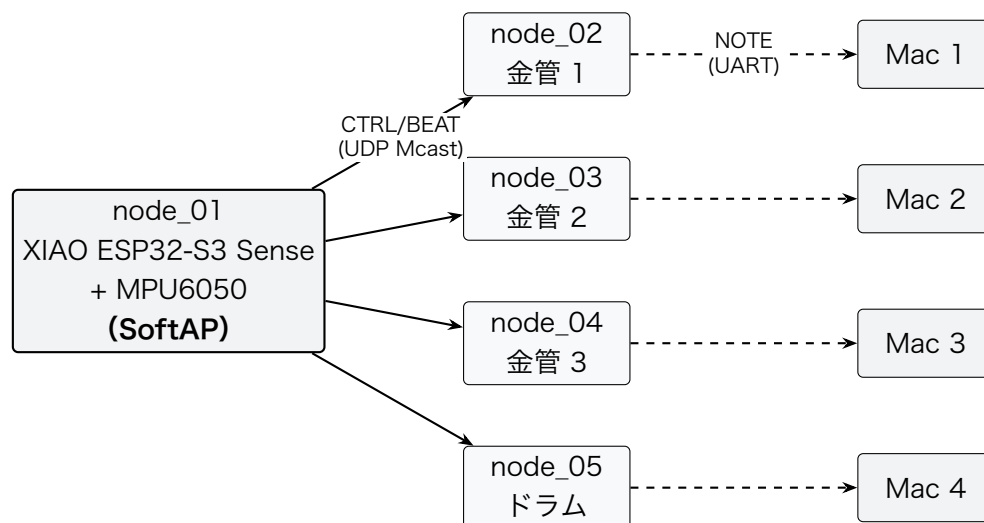


図 1.1 全体ブロック図（指揮者 **node_01** が **SoftAP** を兼ね, 楽器 4 台が **STA** 接続. 実線 = WiFi UDP マルチキャスト, 破線 = 楽器 → Mac の 1 対 1 USB Serial）

1.2.2 ノード役割分担

各ノードの責務とコード担当者を表 1.2 に示す. **node_02~05** の Arduino コードは **共通ソース + パート設定差し替え**（ProjectConfig と楽譜データのみ差分）で構成し, 4 台分の別コードは書き分けない (§2.4.3).

表 1.2: ノード役割分担

ノード	物理配置	主な責務	コード担当
node_01	指揮者の手・指揮棒	MPU6050 読み取り → 拍・テンポ・強弱推定 → CTRL/BEAT 送出. 自身が SoftAP を起動 し楽器を収容	塩澤
node_02	金管 1	BEAT 同期で楽譜進行 → NOTE 送出 (金管 1 パート)	塩澤
node_03	金管 2	BEAT 同期で楽譜進行 → NOTE 送出 (金管 2 パート)	塩澤
node_04	金管 3	BEAT 同期で楽譜進行 → NOTE 送出 (金管 3 パート)	塩澤
node_05	ドラム	BEAT 同期で楽譜進行 → NOTE 送出 (ドラム)	塩澤
Mac × 4	楽器ごとに 1 台	UART で NOTE を受信 → 倍音・ADSR 調整 → 波形生成 → 内蔵スピーカ出力	梅澤 (参考)

1.2.3 データフロー

演奏中の 1 拍ぶんの情報の流れを擬似シーケンスで示す.

1 拍ぶんのデータフロー (演奏中)

- (1) MPU6050 (I2C) → node_01: 加速度・角速度サンプル (200 Hz).
- (2) node_01 内: applyPattern() がフィルタ・拍検出・テンポ推定を実行.
- (3) 振り下ろしを検出: node_01 (SoftAP) → node_02~05 に **BEAT** (beatNo, timestampMs) を UDP マルチキャスト.
- (4) node_02~05 内: 受信 BEAT の beatNo から楽譜位置を算出 → NoteOn フラグ設定.
- (5) node_02~05 → 各 Mac (1 対 1) : **NOTE** (noteNumber, velocity, durationMs) を USB Serial (Serial.write) で送信.
- (6) 並行して node_01 は 20 Hz で **CTRL** (BPM・velocity・state) を同じマルチキャストグループへ送信し続ける.

1.2.4 採用アーキテクチャ (EMA) の方針

全 5 ノードのファームウェアは EMA に準拠する. EMA の中核ルールを本ハッカソンに当てはめた要旨を以下に示す. バイト単位のパケット定義や具体コードは第 2 章で示す.

- ・ **IModule インターフェース**: 全ハードウェアモジュールが `init()` / `updateInput()` / `updateOutput()` / `deinit()` の 4 メソッドを実装する.
- ・ **3 フェーズ実行モデル**: `loop()` は「入力 → ロジック `applyPattern()` → 出力」の順で必ず実行する.
- ・ **SystemData 集約**: モジュール間共有状態を一元化する構造体. モジュール同士の直接呼び出しは禁止.
- ・ **ProjectConfig 集約**: 各モジュールの設定を `{MODULE}_CONFIG` インスタンスとして 1 ヘッダに集約.
- ・ **判断ロジックの位置づけ**: 拍検出・テンポ推定・楽譜進行のような「ハードウェアに触れない判断ロジック」は `IModule` の派生にせず, `applyPattern()` 関数内に書く.

1.3 操作インターフェース設計

本節では「操作インターフェース」を、**ユーザーが触れる物理 IF**（指揮棒・LED）と**ノード間／PC 間の通信プロトコル IF**（CTRL/BEAT/NOTE）の二層に分けて定義する. 通信は **Arduino-Arduino 間 = WiFi UDP**, **Arduino-PC 間 = USB Serial** の二系統に分け, それぞれ役割と性質が異なる. バイト単位のパケットレイアウトは §2.3.3 で具体化する.

1.3.1 ユーザー操作

- ・ **指揮棒（指揮者）**: MPU6050 を搭載した `node_01`（XIAO ESP32-S3 Sense）を指揮棒として手に持ち, 通常の指揮動作（4 拍子・3 拍子の振り下ろし）で操作する. 特別なボタン等は持たない.
- ・ **起動順**: 楽器ノード（`node_02`～`05`）を先に起動し（Idle 状態で SoftAP を待機）, その後指揮者ノード（`node_01`）を起動する. 指揮者の SoftAP 起動で楽器が WaitStart に遷移し, 指揮者の Conducting 入りで BEAT が流れ始めて楽器が Playing に遷移する.
- ・ **Calibrating**: 指揮者ノードは起動後, 最初の 2 秒間に静止時の加速度ノルム（ $\approx$ 重力 1 g）を平均して記録する. 使用者は単に「起動後 2 秒間, 腕を自然に下ろした状態で待つ」だけでよい.

1.3.2 LED 表示仕様

各ノードは内蔵 LED（LED_BUILTIN）の点滅パターンで状態を提示する（表 1.3）. 点滅周期は `StatusLedConfig.blinkIntervalMs` で調整可能.

表 1.3: LED 表示パターン

ノード	状態	LED パターン	意味
node_01	Idle	低速点滅 (1 Hz)	起動直後・SoftAP 起動前
node_01	Calibrating	中速点滅 (2 Hz)	停止時ノルムの学習中 (2 秒)
node_01	Conducting	点灯	通常演奏中
node_01	Fallback	高速点滅 (5 Hz)	IMU エラー or SoftAP 異常
node_02-05	Idle	低速点滅 (1 Hz)	起動直後・SoftAP 未接続
node_02-05	WaitStart	中速点滅 (2 Hz)	SoftAP 接続済, 最初の BEAT 待ち
node_02-05	Playing	点灯	通常演奏中 (次 BEAT が長く来なくても Playing に留まる)

1.3.3 通信プロトコル方針 (CTRL / BEAT / NOTE)

3 種類のメッセージを使い分ける (表 1.4).

表 1.4: 3 種類のメッセージの設計方針

種別	送信元 → 宛先	送信契機	性質	役割
CTRL	node_01 node_02-05 Mcast)	→ 定期 (20 Hz) (UDP	冪等	BPM・velocity・state を継続同期
BEAT	node_01 node_02-05 Mcast)	→ 拍検出時 (不 (UDP 定期)	即時性重視	楽譜ポイント前進トリガ
NOTE	node_02-05 → 各 Mac (USB Serial, 1:1)	楽譜イベント 発火時	順序保証	自パート Mac への 発音依頼

設計原則:

- ・ **CTRL は冪等**: 取りこぼしてもすぐ次が来るため, パケロス耐性は通信方針側で確保される.
- ・ **BEAT は即時**: 1 拍 1 発が原則. 確実性を上げるため同一 beatNo を beatRedundancy 回 (既定 2) 連送し, 受信側で beatNo 重複排除する (冪等処理).

- ・ **BEAT が来ない間も留まる**: 楽器は仮想 BEAT を内挿して走り続けず, BEAT が長く来なくても Playing に留まって次の BEAT 到来時に自然再開する. 連送 (beatRedundancy) と過去 BEAT の即発火で取りこぼしを吸収する.
- ・ **NOTE は単一経路**: 楽器 → 自パート Mac で 1 系統. 楽譜に従って必ず送出する. Arduino-Mac 間は 1 対 1 USB Serial 直結のため, パケロスやネットワーク輻輳の心配なく確定到達するのが利点.

1.3.4 通信方式 (WiFi UDP + USB Serial の二系統)

通信仕様を表 1.5 に示す. 運用時に変わる値 (SSID / パス) は OrcNetConfig のリテラルで一元管理する.

表 1.5: 通信仕様

項目	設計
ノード間 (CTRL/BEAT)	WiFi UDP. node_01 → node_02-05 へ UDP/5001 マルチキャスト (仮 239.0.0.1). 楽器側は <code>WiFiUDP.beginMulticast()</code> でグループに join
WiFi モード	node_01 が SoftAP (<code>WiFi.softAP()</code>), node_02-05 が STA (<code>WiFi.begin()</code>) として接続. 外部ルータ・テザリング不要
SSID / パス	仮 OrchestraAP / orchestra2026 (node_01 が公開し, node_02-05 が同設定で接続). 本番運用で差替え
IP 割り当て	node_01 は SoftAP の既定 192.168.4.1. node_02-05 は SoftAP 内蔵 DHCP で 192.168.4.2--5 を自動取得
Arduino-Mac (NOTE)	間 USB Serial (CDC, Serial.write) . 各楽器ノードと専用 Mac が 1 対 1 で USB 接続. Mac 側 Processing は対応する 1 ポートだけを開く
ボーレート	115200 bps. USB CDC ではボーレート設定は実質無視される (<code>NoteSenderConfig.baudRate</code> に形式上の値として 115200 を入れておくだけで, 通信速度には影響しない)
フレーミング	全パケットの先頭 12 B 共通ヘッダの <code>magic = 0x4F52</code> がフレーム同期マークを兼ねる (§2.3.3)
エンディアン	リトルエンディアン (ESP32-S3 Xtensa LX7 / RA4M1 Cortex-M4 と同一)

1.3.5 パケロス・ジッタ対策

- ・ **CTRL 取りこぼし**: 冪等かつ 20 Hz 定期送信なので設計上問題にならない.
- ・ **BEAT 取りこぼし**: 同一 BEAT を beatRedundancy 回 (既定 2) 連送し, 楽器側で beatNo の重複を排除する. 楽譜進行は beatNo 駆動 (§2.4.3) のため, 連送が両方落ちて 1 拍ぶん欠落しても, その次の BEAT の beatNo から算出される楽譜位置は欠落ぶんだけ正しく進んでおり自己修復する. BEAT が長く来なくても楽器は Playing に留まり, 次の BEAT 到来時に自然再開する (仮想 BEAT を内挿して走り続ける方式は, 指揮者の再起動中などに楽器だけが暴走する危険があるため採らない).
- ・ **ジッタ (楽器間同期誤差)**: BEAT 受信時刻ではなく**マスタ時刻 playAtMasterMs で発音タイミングを揃える** (§2.3.3.6). WiFi の到達時刻ばらつきは時計オフセット推定の誤差に押し付けられ, 同期収束後の定常状態では 4 台の発音差は設計目標の 20 ms 以内に収まる見込み. ただし「鳴らない」を最優先して過去時刻の BEAT も即発火するため, ある 1 台だけ受信が大きく遅れた拍はその台の発音だけが一時的にずれる (詳細と条件の切り分けは §2.3.3.6).
- ・ **プロトコル不整合**: magic / version 不一致のパケットは破棄し, StatusLedModule が data.orcNet を読んで点滅パターンに反映する.

1.4 状態遷移図

各ノード種ごとの演奏状態を 4 状態の有限状態機械として定義する. 状態は ConductorState (指揮者) と PerformerState (楽器) の enum class として SystemData の中に保持し, applyPattern() が遷移を判定する.

1.4.1 指揮者ノード (node_01)

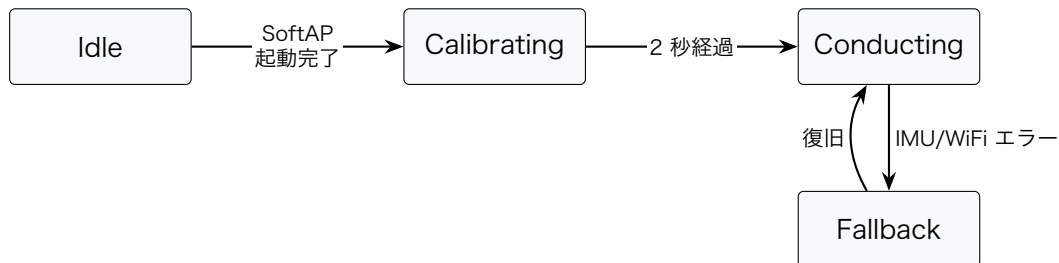


図 1.2 指揮者ノードの状態遷移

表 1.6: 指揮者ノードの状態定義

状態	意味	動作
Idle	起動直後, SoftAP 起動前	ImuModule.updateInput は動くが拍検出は無効, OrcSenderModule は何も送らない
Calibrating	初期静止状態の学習中 (2 秒)	applyPattern() が生加速度ノルムの平均を停止時ノルムとして記録
Conducting	通常演奏中	全モジュール稼働, BEAT を拍検出時に送出, CTRL を 20 Hz 周期で送出
Fallback	エラー発生	CTRL は state=Fallback で送り続ける, BEAT は停止, LED 高速点滅

1.4.2 楽器ノード (node_02~05)



図 1.3 楽器ノードの状態遷移 (3 状態構成)

表 1.7: 楽器ノードの状態定義

状態	意味	動作
Idle	起動直後, node_01 SoftAP 未接続	受信モジュールは動くが楽譜進行ロジックは無効, SoftAP に接続できるまでリトライ
WaitStart	SoftAP 接続済, 最初の BEAT 待ち	CTRL/BEAT を受信し時計オフセットの EMA 学習を開始する. 時計同期の収束は Playing 遷移条件にしない (収束待ちで遷移をブロックすると音が出なくなりやすいため). 最初の数拍は受信遅延ぶんのズレが残るが, CTRL を数発受けたあとに offsetMs が自然収束する

状態	意味	動作
Playing	通常演奏中	BEAT 受信ごとに、その beatNo から楽譜位置 $((\text{beatNo} - \text{startBeatNo}) \bmod \text{kScoreLength})$ を算出して NOTE を送出。BEAT が長く来なくても Playing に留まり、次の BEAT で自然再開する

第 2 章 システムの詳細設計

本章では、第 1 章で示した基本設計を実装可能な水準まで具体化する。塩澤担当範囲（共通層 API・指揮者ノード・楽器ノードの 3 領域）を中心に、部品・ハードウェア配線・ソフトウェア構造・処理フロー・テストを順に記す。コード例は要点抽出にとどめ、フル実装は本書スコープ外とする。

2.1 部品一覧

製品分解構造（PBS）として、本ファームウェアが直接関わるハードウェア・ソフトウェア構成要素を表 2.1・表 2.2 に列挙する。PC 側 Processing 環境やスピーカは本書スコープ外であるが、NOTE パケットの受信境界として記述する。

表 2.1: ハードウェア部品一覧（PBS）

区分	要素	数量	備考
P1.1 指揮者	XIAO ESP32-S3 Sense	1	Espressif ESP32-S3R8（Xtensa LX7 デュアルコア，8 MB PSRAM/Flash）。OV2640 / PDM マイクは本案件未使用（将来拡張余地）
P1.1	MPU6050 (GY-521 モジュール)	1	6 軸 IMU（加速度 $\pm 2/4/8/16$ g + 角速度 $\pm 250/500/1000/2000$ dps）。本案件では ± 4 g / ± 2000 dps。I2C アドレス 0x68。GY-521 はピンヘッダ・LDO・プルアップを実装したブレイクアウト基板で、3.3 V / GND / SDA / SCL / ADO のみ配線すれば動作する
P1.1	内蔵 WiFi (ESP32-S3 直制御)	1	WiFi.h (Arduino-ESP32) で SoftAP + UDP。ESP32-S3 内蔵 WiFi を直接制御するため、WiFi スタックを別チップ越しに扱う必要がない
P1.1	状態 LED	1	LED_BUILTIN (XIAO 基板のユーザ LED) 流用

区分	要素	数量	備考
P1.2 楽器	Arduino UNO R4 WiFi	4	node_02-05 同一 H/W. 楽器構成は 金管 1 / 金管 2 / 金管 3 / ドラム . 主旋律 3 パート (金管) を先に作り込み, ドラム node_05 はその後に実装する
P1.2	内蔵 WiFi	4	WiFiS3, node_01 SoftAP に STA として接続. RA4M1 と内蔵 ESP32-S3 の 2 チップ構成で, 両者は内部の UART で連結される
P1.2	状態 LED	4	LED_BUILTIN 流用
P2 ネットワーク	(node_01 SoftAP で兼用)	0	外部 AP・ルータ・テザリングは使用しない. node_01 が WiFi.softAP() で AP を起動し, 楽器が STA で接続する
P3 給電・通信	USB Type-C ケーブル	5	楽器 4 本は 各 Mac へ 1 対 1 直結 (給電 + Serial 通信兼用). 指揮者ノードは USB ハブ経由で給電のみ
P4 境界 (参考)	Mac (Processing)	4	本書スコープ外. 各楽器から USB Serial で NOTE を受信し, 倍音・ADSR 調整・波形生成を行う. 楽器ごとに 1 台を前提とする. 台数が揃わない場合は 1 台で 4 ポートを受ける構成も考えられるが, その際は Mac 側 Processing を 4 ポート受信に改修する必要があり (梅澤担当・本書スコープ外), Arduino 側の設計は変わらない
P4 境界 (参考)	内蔵スピーカ	4	本書スコープ外. Mac 1 台あたり 1 系統

表 2.2: ソフトウェア技術スタック

レイヤ	採用技術	バージョン / 備考
MCU (指揮者)	XIAO ESP32-S3 Sense	ESP32-S3R8 (Xtensa LX7 240 MHz × 2 / 8 MB PSRAM / 8 MB Flash)

レイヤ	採用技術	バージョン / 備考
MCU (楽器)	Arduino UNO R4 WiFi	Renesas RA4M1 (Cortex-M4 48 MHz) + ESP32-S3 (WiFi 担当)
フレームワーク 言語	Arduino Framework C++ (C++11)	PlatformIO 経由 arduino-core 範囲
ビルド (指揮者)	PlatformIO	platform=espressif32, board=seeed_xiao_esp32s3
ビルド (楽器)	PlatformIO	platform=renesas-ra, board=uno_r4_wifi
外部ライブラリ (指揮者)	Wire (標準同梱)	I2C ドライバを直接叩いて MPU6050 の PWR_MGMT_1/ACCEL_CONFIG/GYRO_CONFIG/ACCEL_XOUT_H を読み書き. Adafruit_MPU6050 等の高水準ライブラリは取り回しの軽さを優先して使わない
WiFi スタック (指揮者)	WiFi.h (Arduino-ESP32 / ESP-IDF)	SoftAP + UDP マルチキャスト/ブロードキャストをネイティブサポート
WiFi スタック (楽器)	WiFiS3	標準同梱. STA 接続 + UDP 受信
設計パターン	EMA	全章前提 (ADR-0005)
共通層 (流用)	ModuleCore (IModule + ModuleTimer)	EMA からそのまま流用. HW 差は ImuModule と OrcNetModule に局在
自作層 (新規)	OrcProtocol OrcNetModule	/ §2.3.3 で定義. ESP32-S3 / RA4M1 でビルド分岐
ネットワーク	UDP over WiFi (ノード間) + USB Serial (PC 間)	二系統に分離

2.2 ハードウェア設計

2.2.1 配線・ピン

指揮者ノード (node_01: XIAO ESP32-S3 Sense) は外付け MPU6050 を I2C で接続する. XIAO の I2C 既定ピンは SDA=D4 (GPIO5), SCL=D5 (GPIO6). Sense 拡張機能の PDM マイクは D11/D12 を専有するが本案件では未使用. 3.3 V / GND を MPU6050 に給

電し、AD0 を GND に落として I2C アドレスを 0x68 に固定する。楽器ノード (node_02-05: UNO R4 WiFi) は内蔵 LED 以外に外部配線を要しない (表 2.3)。

表 2.3: 配線・ピンアサイン

ノード	用途	ピン
node_01	I2C SDA (MPU6050)	D4 (GPIO5, I2C_SDA_PIN = 5)
node_01	I2C SCL (MPU6050)	D5 (GPIO6, I2C_SCL_PIN = 6)
node_01	MPU6050 電源	3V3 / GND. AD0 を GND に落とし I2C アドレスを 0x68 に固定
node_01	状態 LED	LED_BUILTIN (XIAO ユーザ LED)
node_01	給電	USB Type-C (USB ハブ経由で給電. 本番では Serial 不使用, 開発・試験時のみデバッグ Serial を読む)
node_02-05	状態 LED	LED_BUILTIN
node_02-05	給電 + NOTE 送信	USB Type-C (各楽器が専用 Mac へ 1 対 1 直結 . Serial.write で NOTE 出力)

2.2.2 ネットワーク構成

通信は二系統に分離する。Arduino 同士は WiFi UDP (node_01 自身が立てる SoftAP 配下) で、Arduino と Mac は USB Serial 直結で通信する。node_01 (XIAO ESP32-S3 Sense) は ESP-IDF を介した `WiFi.softAP("OrchestraAP", "orchestra2026")` で SoftAP を起動し、node_02-05 (UNO R4 WiFi) は `WiFiS3` で同 SSID/パスへ STA として接続する。SoftAP の既定アドレスは 192.168.4.1、楽器側は内蔵 DHCP で 192.168.4.2--5 を取得する。node_01 は CTRL/BEAT を **UDP マルチキャスト** (仮 239.0.0.1:5001) で送信し、node_02-05 はそれぞれグループに join して受信する。NOTE は各楽器ノードが **専用 Mac のシリアルポート** へ `Serial.write` で書き込む (楽器 ↔ Mac は 1 対 1)。本番運用時の SSID / パスは `OrcNetConfig` のリテラルで切替可能とする。Mac 側の Processing は自分に繋がっているシリアル 1 ポートだけを開いて受信する。

採用判断: 指揮者ノードだけを XIAO ESP32-S3 にしたのは、ESP32-S3 を Arduino-ESP32 で直接制御すると `WiFi.softAP()` と `WiFiUDP` (マルチキャスト含む) がネイティブにサポートされ、内蔵 WiFi を別チップ越しに扱う UNO R4 WiFi より SoftAP の起動・送受信の制御自由度が高いためである (ESP-NOW などの代替手段もストレッチ拡張として選べる)。

利点: 外部ルータ・テザリングが不要なため、会場 LAN のセキュリティ制限や電波混雑の影響を受けない。

STA 側の注意: AP は省電力モードの STA がいると、グループ宛（ブロードキャスト／マルチキャスト）フレームの配送が DTIM ビーコン周期ぶん遅れる（または取りこぼす）。これを避けるため、楽器ノード（STA 側）の WiFi 省電力モードは無効化して運用する。WiFiS3 でこれを無効化する具体的な手段は実機で確認するが、もし無効化できなかった場合は、beatLookaheadMs を 100-150ms に広げて配送遅延ぶんを吸収する（振り→音の固定遅延は増えるが演奏体験上は許容範囲）、もしくは指揮者側を CTRL/BEAT のユニキャスト連送に切り替える、のいずれかを代替案とする (§2.5.3)。

指揮者ノードの機能集中: 指揮者ノードは SoftAP（ビーコン送信・STA 管理・DHCP）・IMU の 200Hz サンプリング・拍検出・CTRL/BEAT 送信を 1 チップ（ESP32-S3）で回す。SoftAP/WiFi の処理は ESP-IDF のタスク／ISR 側で走るため、applyPattern() の所要時間や入力フェーズ所要時間 (§2.5.1) には現れないが、IMU の I2C 読みがビーコン処理に割り込まれてサンプリング間隔が乱れうる。設計としては、IMU の実サンプリング間隔のジッタ（公称 5ms 周期に対するばらつき）を許容範囲に保ち実機で監視する（乱れが大きければサンプリング周期や SoftAP のビーコン間隔を見直す）。

2.3 ソフトウェア設計

2.3.1 ファイル構成

firmware/ 配下は **共通層**と**ノード別プロジェクト**の 2 段構成にする。EMA に準拠し、PlatformIO の lib_extra_dirs で共通層を全ノードから参照する。

firmware/ ディレクトリ構成（抜粋）

```
firmware/
|-- common/
|   |-- lib/                                # 全ノード共通 (lib_extra_dirs で参照)
|       |-- ModuleCore/                    # コア層 (プロジェクト非依存)
|           |-- IModule.h
|           |-- ModuleTimer.h
|       |-- OrcProtocol/                    # CTRL/BEAT/NOTE のパケット定義
|           |-- OrcProtocol.h
|           |-- OrcProtocol.cpp
|       |-- OrcNetModule/                  # WiFi 接続維持 + UDP 送受信
|           |-- OrcNetModule.h
|           |-- OrcNetModule.cpp
|       |-- StatusLedModule/               # 出力: 状態 LED (全ノード共有)
|       |-- SerialDebug/                  # SERIAL_DEBUG 切替のデバッグ出力マクロ
|-- node_01/                               # 指揮者ノード
|   |-- platformio.ini
|   |-- include/
|       |-- SystemData.h                  # {Module}Data を集約
|       |-- ProjectConfig.h              # {MODULE}_CONFIG + 共有バスピン
```

```

| |-- src/
| | |-- main.cpp                # 入出力配列・3 フェーズループ
| | |-- applyPattern.cpp        # フィルタ/拍検出/テンポ推定/状態遷移
| |-- lib/
| | |-- ImuModule/              # 入力: IMU
| | |-- OrcSenderModule/        # 出力: CTRL/BEAT 送信
|-- node_02/                    # 楽器ノード (金管 1)
| |-- platformio.ini
| |-- include/
| | |-- SystemData.h
| | |-- ProjectConfig.h        # PART_ID=0x02, startBeatNo 等
| | |-- score_data.h           # このパートの楽譜 (c 配列)
| |-- src/
| | |-- main.cpp
| | |-- applyPattern.cpp        # 楽譜進行 (index 駆動 + 細分音符予約)
| | |-- score_data.cpp
| |-- lib/
| | |-- OrcReceiverModule/      # 入力: CTRL/BEAT 受信
| | |-- NoteSenderModule/      # 出力: NOTE 送信
|-- node_03/ (構造同じ。ProjectConfig と score_data.* が差分)
|-- node_04/
`-- node_05/ (ドラム)

```

各ノードの platformio.ini には次の設定を含める。-I include は EMA の必須設定 (lib/ から include/SystemData.h を参照するため)、共通層の各モジュールへの-I パスと lib_extra_dirs・lib_ldf_mode = deep+ は、5 ノードで firmware/common/lib/ を共有しつつヘッダの推移的依存まで解決させるための設定。SERIAL_DEBUG は本番 (NOTE バイナリ送信) と開発 (テキストログ) をビルド時に切り替えるフラグ。

```

build_flags =
  -I include                ; lib/ から include/SystemData.h を参照するために必須
  -I ../common/lib/ModuleCore ; 共通層の各モジュールヘッダへのパス
  -I ../common/lib/OrcProtocol
  -I ../common/lib/OrcNetModule
  -I ../common/lib/StatusLedModule
  -I ../common/lib/SerialDebug
  -DSERIAL_DEBUG=0          ; 0=本番 (NOTE バイナリ送信) / 1=開発 (テキストログ)
lib_extra_dirs = ../common/lib ; 共通層を全ノードで共有
lib_ldf_mode = deep+        ; 共通層の推移的なヘッダ依存まで解決させる

```

2.3.2 オブジェクト設計 (クラス図)

EMA の IModule 抽象基底をすべてのハードウェアモジュールが継承する。OrcNetModule は共通層に置き両ノード種で共有する (図 2.1)。

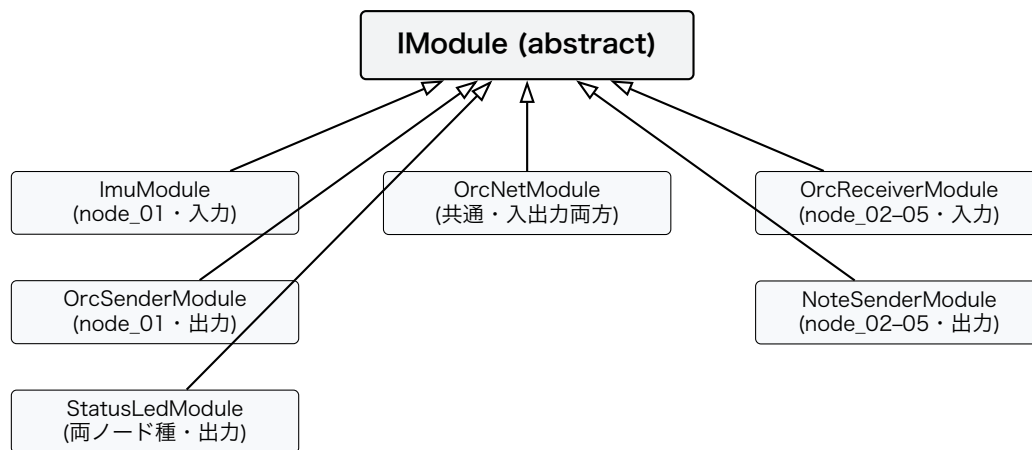


図 2.1 モジュールのクラス継承関係

表 2.4: モジュール一覧（入出力分類別）

モジュール	ノード	分類	責務
ImuModule	node_01	入力	MPU6050 (I2C) の 6 軸を読み data.imu に書く
OrcSenderModule	node_01	出力	data.beat/data.tempo を読み CTRL/BEAT 送信キューに投入
OrcReceiverModule	node_02-05	入力	data.orcNet.lastCtrl/Beat を読み data.receiver に整形
NoteSenderModule	node_02-05	出力	data.noteOut.pendingOn/Off を見て NOTE を Serial.write で PC へ送信
OrcNetModule	node_01・ node_02-05	入出力	WiFi 接続維持, UDP 受信ポーリング (入力) と CTRL/BEAT 送信キューフ ラッシュ (出力)
StatusLedModule	全ノード	出力	状態に応じた LED 点滅パターン表示

2.3.3 関数設計

2.3.3.1 IModule インターフェース（流用）

EMA のすべての入出力モジュールが継承する基底クラス。3 フェーズループから一律に呼ばれる入口を定める。依存を避けるため SystemData は前方宣言で参照する。

IModule (モジュール抽象基底)

メンバ

シグネチャ	役割
enabled	有効/無効フラグ. init() 失敗時に false に落とし、ループ側はこれを見て呼び出しをスキップ
init()	ハードウェア初期化. 成功で true, 失敗で false を返す
updateInput(data)	入力フェーズで呼ばれる. センサ値や受信結果を data に書く
updateOutput(data)	出力フェーズで呼ばれる. data の値をハードウェア／送信に反映
deinit()	後始末. 基底側は空実装

init() の失敗時は setup() 側で MAX_RETRY 回リトライし、それでも失敗した場合は enabled = false とする. ロジックフェーズで定期的に再 init() を試みて enabled = true に復帰させる (§2.4.2 参照).

2.3.3.2 ModuleTimer (流用)

「ある基準時刻からの経過 ms」を返す軽量タイマ. 内部に基準時刻を 1 つ持つだけの薄いラップで、周期判定やタイムアウト判定に使う.

ModuleTimer (経過時間タイマ)

メンバ

シグネチャ	役割
setTime(offsetMs = 0)	基準時刻を「現在 - offsetMs」にセット. 引数省略で「いま」を起点にする
getNowTime()	基準時刻からの経過 ms を返す

用途例: IMU サンプリング 200Hz (intervalMs = 5), CTRL 送信 20Hz (50ms), LED 点滅周期, BEAT 不応期, CTRL 送信周期.

2.3.3.3 通信パケット定義 (OrcProtocol)

3 種のパケットすべてに 12 バイトの共通ヘッダを付与する. データはリトルエンディアンでシリアライズする. CTRL/BEAT は WiFi UDP マルチキャスト, NOTE は USB Serial (バイナリ列を直接書込) で転送する. magic = 0x4F52 は WiFi UDP ではプロトコル識別

子, USB Serial では **フレーム同期マーク**を兼ねる. Mac 側はバイトストリームを 1 バイトずつ走査し, 先頭 2 バイトをリトルエンディアンで uint16 に組み立てた値が 0x4F52 になった位置をフレーム先頭とみなして固定 20 バイトを 1 パケットとして読む (type で CTRL/BEAT/NOTE を識別するが, USB Serial に流れるのは NOTE のみ).

共通ヘッダ

全 12 バイト. 全パケットの先頭に付与する.

オフセット	フィールド	型	説明
0	magic	uint16_t	固定値 0x4F52 (ASCII "OR")
2	version	uint8_t	初期値 0x01
3	type	uint8_t	1=CTRL, 2=BEAT, 3=NOTE
4	seq	uint32_t	単調増加. 受信側でのパケロス検出に使える予約フィールド (現設計では送付のみ)
8	timestampMs	uint32_t	マスタ時刻 (= 指揮者ノードの millis()). 楽器側は受信時刻と比較してオフセット推定に使う (§2.3.3.6)

CTRL ペイロード

type = 1. 定期 20 Hz 送信. 共通ヘッダ含め全 20 バイト.

オフセット	フィールド	型	説明
12	bpmQ8	uint16_t	BPM × 8 (例: 120.5 → 964)
14	velocity	uint8_t	0-127. ストレッチ未実装時は固定 64
15	state	uint8_t	0=Idle, 1=Calibrating, 2=Conducting, 3=Fallback
16	予約	uint8_t[4]	ゼロ埋め

BEAT ペイロード

type = 2. 拍検出時イベント送信. 共通ヘッダ含め全 20 バイト.

オフセット	フィールド	型	説明
12	beatNo	uint16_t	演奏セッションの曲頭からの拍番号. 曲をループ再生しても 0 に戻さず単 調増加させる (uint16_t, 120BPM で約 9 時間ぶんあり実演では溢れな い). 楽器側はこの値と startBeatNo の差から楽譜位置を算出する
14	予約	uint8_t[2]	ゼロ埋め
16	playAtMasterMs	uint32_t	マスタ時刻でこの ms に発音せ よ という未来時刻指定. 送信時に masterNow + lookahead で算出

NOTE ペイロード

type = 3. 楽譜イベント発火時に USB Serial へ送信. 共通ヘッダ含め全 20 バイト.

オフセット	フィールド	型	説明
12	partId	uint8_t	0x02-0x05 (= node_02-05 の PART_ID. どの楽器/声部かを表す楽 器番号)
13	noteNumber	uint8_t	MIDI ノート番号 (0-127, 60=C4). ドラムパート (partId = 0x05) では GM ドラムマップとして解釈する (36=バスドラム, 38=スネア, 42=ク ローズドハイハット 等)
14	velocity	uint8_t	0-127
15	gate	uint8_t	1=NoteOn, 0=NoteOff
16	durationMs	uint16_t	発音予定長さ (参考値)
18	予約	uint8_t[2]	ゼロ埋め

2.3.3.4 OrcNetModule API (WiFi UDP 専用)

WiFi 接続維持と UDP 送受信ポーリングを集約する IModule 実装. CTRL/BEAT のみを担
当し, NOTE は別モジュール (NoteSenderModule) が USB Serial で直送する. 入出力両方
の責務を持つため, inputModules[] と outputModules[] の両方に登録する.

WifiMode (列挙)

値	意味
---	----

SoftAp	自身がアクセスポイントを起動する側 (指揮者ノード node_01)
Sta	既存 SoftAP に接続する側 (楽器ノード node_02-05)

OrcNetConfig (生成時に渡す設定)

フィールド	役割
-------	----

mode	動作モード. SoftAp (指揮者) / Sta (楽器)
ssid	SoftAp なら自身が公開する SSID. Sta なら接続先 SSID
pass	WPA2-PSK パスフレーズ
multicastIp	CTRL/BEAT 配信先マルチキャスト IP (例: 239.0.0.1)
udpPort	CTRL/BEAT 共通ポート (既定 5001)
channel	SoftAp 時のみ参照 (既定 6)
reconnectIntervalMs	切断検出後の再接続試行間隔 (既定 2000 ms)

OrcNetData (SystemData.orcNet に置く受け渡し領域)

受信側 (他モジュールが読む)

フィールド	役割
-------	----

wifiConnected	WiFi 接続が確立しているか
hasNewCtrl	今フレームで新しい CTRL を受信したか (読み手が消費後にクリア)
hasNewBeat	今フレームで新しい BEAT を受信したか (同上)
lastCtrl	直近に受信した CTRL ペイロード
lastBeat	直近に受信した BEAT ペイロード

送信側 (送信したいモジュールが書く)

フィールド	役割
hasPendingCtrl	未送信 CTRL がある（送出後に OrcNetModule がクリア）
pendingCtrl	送信したい CTRL ペイロード
hasPendingBeat	未送信 BEAT がある（同上）
pendingBeat	送信したい BEAT ペイロード
pendingBeatRedundancy	同一 BEAT を何発まで連送するか。フィールドの初期値は 1 だが、送信側（OrcSenderModule）が beatRedundancy（既定 2）の値を入れて上書きするため、実行時は 2 になる

OrcNetModule (クラス)

シグネチャ	役割
OrcNetModule(config)	コンストラクタ。OrcNetConfig を保持
init()	SoftAp なら WiFi.softAP(), Sta なら WiFi.begin() の後に Udp.beginMulticast() を呼ぶ
updateInput(data)	WiFi 状態を data.orcNet.wifiConnected に反映。UDP 受信をポーリングして lastCtrl/lastBeat と hasNewCtrl/hasNewBeat に書く
updateOutput(data)	hasPendingCtrl/Beat が立っていれば該当パケットをマルチキャスト送出。pendingBeatRedundancy 回連送した後にフラグをクリア
deinit()	WiFi / UDP 後始末

ルール (EMA「モジュール間直接呼び禁止」に準拠)： 送受信ともに OrcNetModule の公開メソッドを他モジュールから直接呼ばない。送信側は OrcSenderModule（楽器側は applyPattern()）が data.orcNet.pendingCtrl/pendingBeat と hasPendingCtrl/Beat を立てるだけで、実際の Udp.send() は OrcNetModule.updateOutput() が一括処理する。受信側も tryRecvXxx() のような外向き API を持たず、OrcNetModule.updateInput() が lastCtrl/lastBeat と hasNewCtrl/hasNewBeat に書き込み、他モジュールはこれを読むだけ。モジュール間の結合は SystemData の一点に閉じる。

2.3.3.5 NoteSenderModule API (USB Serial 専用)

楽器ノード (node_02-05) が NOTE をハードウェア Serial へ書き出すための IModule 実装. `init()` で `Serial.begin(baudRate)` を呼び、`updateOutput()` で `data.noteOut.pendingOn/Off` フラグを見て `NotePacket` を構築 → 20 バイトのバイナリを `Serial.write()` で一括送出する.

NoteSenderConfig (生成時に渡す設定)

フィールド	役割
<code>baudRate</code>	USB CDC ボーレート (既定 115200. CDC では実質無視される形式上の値)
<code>partId</code>	自パート識別子 (<code>PART_ID</code> の値, 0x02-0x05). 1 対 1 USB 接続のため Mac 側では参考値

NoteSenderData (送信制御の内部状態)

フィールド	役割
<code>noteSeq</code>	送信した NOTE の連番. 共通ヘッダの <code>seq</code> に積む
<code>lastSentMs</code>	直近の <code>Serial.write()</code> 完了時刻

NoteSenderModule (クラス)

シグネチャ	役割
<code>NoteSenderModule(config)</code>	コンストラクタ
<code>init()</code>	<code>Serial.begin(baudRate)</code> を実行
<code>updateOutput(data)</code>	<code>data.noteOut.pendingOn/Off</code> を見て <code>NotePacket</code> を組み立て、20 バイトを <code>Serial.write()</code> で一括送出. 送出後にフラグをクリア

Mac 側 Processing は、自パートの楽器ノードに繋がっている 1 つの USB シリアルポートを開き、magic 0x4F52 を探してフレーム同期し、`type=3` のとき NOTE ペイロード 8 バイトを読み取って音色合成 (倍音・ADSR 調整 → 波形生成) に渡す. `partId` は 1 対 1 接続のため検証用途で参照する.

2.3.3.6 マスタクロック方式と時刻同期

WiFi UDP マルチキャストは PHY のベーシックレートで配信されるうえ、楽器ノード (UNO R4 WiFi) は RA4M1 (メイン MCU) と WiFi を担う ESP32-S3 の 2 チップ構成で、両者は内部の UART を介する。WiFi の到達ばらつきにこの内部経路のオーバーヘッドも加わり、4 台がアプリ層で BEAT を受け取る時刻には数 ms~十数 ms のジッタが乗りうる。これをそのまま発音に使うと楽器間同期誤差が運次第になる。本設計ではこのジッタを **時計同期の誤差に押し付ける**ことで、楽器間の相対同期を小さく抑える。

基本アイデア

指揮者ノード (= SoftAP) の millis() を **マスタ時刻** として全ノードで共有する。BEAT は「**マスタ時刻 playAtMasterMs に発音せよ**」という **未来時刻指定**の指示として送る。楽器は自分の millis() をマスタ時刻に変換するためのオフセットを学習し続け、masterNow がその時刻に到達した瞬間に発音する。WiFi の到達時刻がばらついても、4 台が見ている共通のマスタ時刻に揃うため発音タイミングは合致する。

オフセット推定 (楽器側)

共通ヘッダの timestampMs を「送信時のマスタ時刻」と読み替える。楽器は CTRL/BEAT を受信するたびに、自ノード時計とのズレを 1 サンプルとして取り出し、係数 $\alpha = 0.10$ の指数移動平均 (EMA) で offsetMs を更新する。

オフセット推定アルゴリズム

ステップ 処理内容

受信時	sample = pkt.header.timestampMs - millis() (マスタ時刻と自時計の差)
更新	offsetMs $\leftarrow$ offsetMs + $\alpha \cdot (\text{sample} - \text{offsetMs})$ ($\alpha = 0.10$)
発音判定	masterNow = millis() + offsetMs. 保 留 BEAT の playAtMasterMs を超えたら発火

CTRL は 20 Hz で常時流れるためサンプル数は十分で、EMA 数発で収束する。片方向放送のため **WiFi の片道遅延がオフセットに系統誤差として乗る**。この片道遅延が 4 台でほぼ同程度であれば、**相対同期 (楽器間同期誤差で問題になるのはこちら)**は保たれる。ただし各台の内部経路の遅延・ジッタ分布が大きく異なるとこの前提は崩れる。beatRedundancy は到達率を、beatLookaheadMs は待ち時間を変えるだけで台ごとの片道遅延の系統差そのものは消せないため、4 台の同期誤差を実機で確認し、系統差が無視できなければ往復測定 (楽器が即 ACK を返し、指揮者で往復時間を測る) で各台の片道遅延を個別に較正し、その固定補正値を offsetMs に足し込む方式に拡張する (§2.5.3)。

lookahead と過去 BEAT の扱い

指揮者は BEAT 送出時に $\text{playAtMasterMs} = \text{masterNow} + \text{beatLookaheadMs}$ を計算する (beatLookaheadMs は `ORC_SENDER_CONFIG` で集約, 仮値 50 ms). 楽器側は次の挙動で安全側に倒す:

- ・ **未来時刻** ($\text{playAtMasterMs} > \text{masterNow}$): 通常, masterNow 到達まで待機.
- ・ **過去時刻: 即発火 (捨てない)**. 受信遅延が大きく指定時刻を過ぎていても捨てない. 捨てる方式 (猶予を超えたらスキップ) は, 指揮者の WiFi リトライや楽器の起動直後に「音が鳴らない」原因になりやすいためである. 代わりに $\text{beatRedundancy} = 2$ で同一 BEAT を冗長送信し, どれか 1 発が lookahead 内に届けば「ほぼ揃う」状態を作る. なお, 遅れて届いた古い beatNo で発火すると算出される楽譜位置が一瞬巻き戻るが, 次に届く新しい beatNo で即座に正しい位置へ復帰する (楽譜進行は beatNo 駆動, §2.4.3).

この方式は「鳴らない」を最優先した best-effort 同期である. **同期収束後の定常状態**——時計オフセットが EMA で収束し, 全台がどれか 1 発の BEAT を $\text{waitMs} > 0$ (未来時刻) で受信できている拍——では, 楽器間の発音タイミング差は設計目標の 20 ms 以内に収まる見込みである. 一方, ある 1 台だけ受信が lookahead (50 ms) を大きく超えて遅れた拍は, その台の発音だけが一時的にずれる (「鳴らない」を避けるための例外措置という位置づけ). すなわち設計目標の 20 ms はこの定常状態を狙う値であり, 遅延無制限の即発火と矛盾するものではない. 実機で楽器間同期誤差を計測し, 許容範囲に収まらなければ lookahead を広げる・連送回数を増やす等で調整する.

同期確立フェーズ

楽器ノードの `WaitStart` → `Playing` 遷移条件は「**最初の BEAT を受信したら**」とする. 時計同期が収束してから `Playing` に入る設計は, CTRL 数発の収束待ちで「鳴らない」原因になりやすいため採らない. 最初の数拍は受信遅延ぶんのズレが残るが, CTRL を数発受けたあと offsetMs が EMA で自然収束する. 楽器間同期誤差の評価は同期収束後の定常状態で行う (`clockSyncMinSamples` は収束判定の目安で, 状態遷移条件には使わずデバッグ表示にのみ用いる).

設計上の固定遅延 (受け入れ判断)

$\text{beatLookaheadMs} = 50 \text{ ms}$ は「指揮の振り下ろしから音が出るまでの固定遅延」としてそのまま乗る. 映像 (指揮動作) に対して音が遅れる方向のズレは比較的許容されやすく, 50 ms 程度は演奏体験上ほぼ問題にならない範囲と考える. **ハッカソン用途では受け入れる設計判断**とする. 振り下ろし最下点予測などの高度化は本書スコープ外.

2.3.3.7 命名規則・ログ書式 (EMA 準拠)

- ・ Config 型: `{Module}Config` (例: `ImuConfig`)

- ・ Config インスタンス: {MODULE}_CONFIG (例: IMU_CONFIG)
- ・ Data 型: {Module}Data (例: ImuData)
- ・ ログ: [ModuleName] msg 形式 (例: [OrcNet] reconnecting (attempt 3))
- ・ デバッグログは SERIAL_DEBUG ビルドフラグ (platformio.ini で 0/1) で切替え可能 (SerialDebug.h のマクロ群)

2.4 処理フローの設計

2.4.1 共通: 3 フェーズ実行モデル

全ノード共通で, loop() は「入力 → ロジック applyPattern() → 出力」の 3 フェーズを毎周期この順で実行する. enabled == false のモジュールはスキップする.

loop() の実行順序

フェーズ	処理内容
1. 入力	inputModules[] を先頭から順に updateInput(systemData) で呼び出す. センサ値や UDP 受信結果が SystemData に書き込まれる
2. ロジック	applyPattern(systemData) を 1 度だけ呼ぶ. フィルタ / 拍検出 / 状態遷移 / 楽譜進行などはすべてここに集約
3. 出力	outputModules[] を先頭から順に updateOutput(systemData) で呼び出す. LED 表示・UDP 送信・Serial 送出自が反映される

入力モジュールは SystemData を書き込み, 出力モジュールは読み込む. クロス参照は禁止. OrcNetModule は両配列に登録し, 「入力フェーズで先に受信, 出力フェーズの最後で送信」を配列順で制御する.

2.4.2 指揮者ノード (node_01)

2.4.2.1 モジュール構成

- ・ 入力配列: OrcNetModule, ImuModule
- ・ 出力配列: OrcSenderModule, StatusLedModule, OrcNetModule
- ・ ロジック: applyPattern() (フィルタ・拍検出・テンポ推定・状態遷移)

2.4.2.2 ProjectConfig (抜粋)

node_01/include/ProjectConfig.h に、共有バスピンと各モジュール設定インスタンス、ロジック層パラメータを集約する。具体値はこの 1 ファイルでだけ管理し、モジュール本体にはハードコードしない。

共有バスピン

定数	値	役割
I2C_SDA_PIN	5 (D4 = GPIO5)	XIAO ESP32-S3 Sense の I2C SDA 既定
I2C_SCL_PIN	6 (D5 = GPIO6)	同上 SCL

IMU_CONFIG

キー	値	役割
address	0x68	MPU6050 I2C アドレス (AD0=GND). AD0=VCC なら 0x69
sampleIntervalMs	5	200 Hz サンプリング
accelRangeG	4	加速度フルスケール (2/4/8/16 g 選択可)
gyroRangeDps	2000	ジャイロフルスケール (250/500/1000/2000 dps 選択可)

ORC_NET_CONFIG (指揮者)

キー	値	役割
mode	SoftAp	node_01 自身が AP を起動
ssid	"OrchestraAP"	公開 SSID
pass	"orchestra2026"	WPA2-PSK
multicastIp	239.0.0.1	CTRL/BEAT 配信先
udpPort	5001	共通ポート
channel	6	SoftAp チャンネル

ORC_SENDER_CONFIG

キー	値	役割
ctrlIntervalMs	50	CTRL 送信周期 (20Hz)
beatRedundancy	2	同一 BEAT の連送回数 (lookahead 期限切れ回避)
beatLookaheadMs	50	マスタ時刻で何 ms 先を発音指定にするか

STATUS_LED_CONFIG

キー	値	役割
pin	LED_BUILTIN	表示ピン
blinkIntervalMs	500	既定点滅周期

LOGIC_PARAMS (applyPattern()) のロジック係数, 主なもの)

キー	値	役割
lpfAlpha	0.10	加速度 IIR LPF の係数
beatDynThresholdG	1.20	Armed 突入トリガ. 動加速度ノルム (LPF 後の加速度ノルム - キャリブで採った停止時ノルム $\approx 1g$) の閾値. 重力相当の下駄を引いた「純粋な振りの加速度強度」で判定するため校正姿勢に依存しない
beatFirePathM	0.20	早期発火閾値. Armed 中の擬似経路長 (動加速度を 2 回時間積分した近似変位) がこの値に達した瞬間に拍を発火
beatRefractoryMs	350	拍検出後の不応期 (約 170 BPM 上限). 1 振り中の振り戻しを 2 拍目として誤検出しないため
bpmEmaAlpha	0.30	テンポ推定 EMA 係数
bpmMin / bpmMax	40.0 / 240.0	推定 BPM の下限・上限クランプ
calibrationMs	2000	起動時キャリブレーション期間 (停止時の加速度ノルムを平均して保持)
imuTimeoutMs	200	IMU が ready=false のまま継続したら Fall-back 遷移

このほか, Armed の最低保持時間・タイムアウト, リリース判定の絶対/相対閾値とデバウンス時間など, 拍検出ステートマシン内部の細かい係数も LOGIC_PARAMS に集約する (値は実機で調整).

2.4.2.3 SystemData (抜粋)

ImuData・OrcNetData・OrcSenderData・StatusLedData に加え, applyPattern() の中間状態として BeatLogicData・TempoLogicData・CalibrationData・ConductorStateData を集約する.

ConductorState (列挙)

値	意味
Idle	起動直後、未初期化
Calibrating	停止時ノルム採取中（起動から calibrationMs）
Conducting	通常動作、拍検出と CTRL/BEAT 送信を行う
Fallback	IMU / WiFi 異常時のフェイルセーフ

ロジック中間データ

構造体	主なフィールド／役割
BeatLogicData	event (今周期に拍を検出したか) / beatNo (曲頭からの拍番号) / lastBeatMs (直近検出時刻) / pathLenM・gateState・armedPeakDyn (拍検出ステートマシンの可視化用). BEAT パケットの playAtMasterMs は OrcSenderModule 側で masterNow + beatLookaheadMs として算出する
TempoLogicData	bpm (EMA 平滑後の推定 BPM) / nextBeatPredictedMs (次拍予測時刻) / velocity (CTRL に乗せる強度、初期実装は固定 64)
CalibrationData	done (完了フラグ) / startMs (採取開始時刻) / sampleCount・accumNorm (採取中の累積) / gravityMag (停止時の加速度ノルムの平均 $\approx$ 重力 1g。軸別ベクトルではなくスカラー)
ConductorStateData	state (現在の ConductorState)

SystemData (指揮者ノードの集約データ)

メンバ	役割
imu	IMU 読み取り値 (加速度・角速度・ノルム)
orcNet	OrcNetModule の受信結果と送信予約
sender	CTRL/BEAT のシーケンス番号など送信側の内部状態
led	StatusLedModule への指示値
beat	拍検出結果 (BeatLogicData)
tempo	テンポ推定結果 (TempoLogicData)
calibration	キャリブレーション進行状況 (CalibrationData)
conductor	状態遷移の現在状態 (ConductorStateData)

2.4.2.4 applyPattern() の処理フロー

毎周期, 以下の順で処理する.

1. **IIR LPF + ノルム計算 + 動加速度算出**: `data.imu.acc` を一次ローパス ($\alpha = 0.10$) で平滑化し `accLpf` を得て, ノルム `accNorm = |accLpf|` を計算する. これから Calibrating 中に採った**停止時ノルムの平均** `calibration.gravityMag` ($\approx$ 重力 1g) をスカラーとして差し引き, `dynNorm = max(accNorm - gravityMag, 0)` を拍検出の判定対象とする (軸ごとの重力ベクトルではなくノルムで引くので, 校正したときの姿勢に依存しない). Calibrating 完了前は `gravityMag = 0` のため `dynNorm  $\approx$  accNorm`. 経路長の積分には `accLpf` の向きを保ったまま大きさを `dynNorm` にスケールした近似ベクトル `dynAcc` を用いる.
2. **拍検出 (ゲート式ステートマシン + 早期発火)**: `state == Conducting` のときのみ実行. 詳細は次項.
3. **テンポ推定 (EMA)**: 拍間隔 $\rightarrow$ 瞬時 BPM $\rightarrow \alpha = 0.30$ の指数移動平均で `data.tempo.bpm` を更新. `[bpmMin, bpmMax]` にクランプし `nextBeatPredictedMs` を計算.
4. **状態遷移**: Idle $\rightarrow$ Calibrating (SoftAP 起動完了) $\rightarrow$ Conducting (`calibrationMs` 経過) $\rightarrow$ Fallback (IMU/WiFi エラー) $\rightarrow$ Conducting (復旧) の判定. Fallback 判定は `(now - imu.sampleAtMs) > imuTimeoutMs` または `!orcNet.wifiConnected` で発動.
5. **LED 反映**: `conductor.state` に応じて `led.solidOn` と `led.blinkIntervalMs` を設定.

拍検出アルゴリズム

単純な「ノルムピーク閾値 + 不応期」だと, ピーク後に発火して振りと音のラグが大きい, 振り戻しを 2 拍目として拾う, 連続スイングで状態が抜けない, 軽い揺れで誤発火する,

といった問題が出やすい。これらを構造的に避けるため、**ゲート式ステートマシン** (Idle / Armed の 2 状態) で拍を検出する。

拍検出ステートマシン (applyPattern() 内のローカル状態)

- ・ **Idle → Armed**: 動加速度ノルム `dynNorm` が `beatDynThresholdG` (1.20 g) を超えたら Armed に入り、積分用変数・ピーク値・突入時刻をリセットする。Armed 中だけ動加速度を 2 回時間積分して擬似経路長を更新する (Idle では積分しないのでノイズが蓄積しない)。
- ・ **発火 (早期発火)**: Armed 中、擬似経路長が `beatFirePathM` (0.20 m) に達し、かつ前回拍から `beatRefractoryMs` (350 ms) 経過していれば拍を発火する (1 Armed セッションに高々 1 回)。発火後も Armed は維持し、「発火 → 即 Idle → 高い動加速度のまま再 Arm → 不応期境界で再発火」という連続発火を構造的に防ぐ。
- ・ **Armed → Idle (リリース)**: 動加速度が一定値を下回るか、Armed 中ピークの一定割合を割ったら振り終わりと見なす。単発ノイズで抜けないようにデバウンスを掛ける。Armed が長引いたら (タイムアウト) `path/不応期` を満たしていれば拍として採用してから Idle に戻す (取りこぼし防止)。

振る向きへの非依存性: Armed 突入の判定はスカラーの動加速度ノルムで行うため、振る向き (縦・横・斜め) にほぼ依存せずどんな振り方でも拍を拾える (ノルム差方式の性質上、重力に直交する純水平の振りはわずかに過小評価されるが、自然な指揮の振り下ろしには下向き成分があり十分な余裕がある。取りこぼし/誤発火が出たら閾値を 1.0–1.4 g で再調整する)。ただし起動時 `Calibrating` の 2 秒間に静止していないと停止時ノルムの推定がずれるため、使用者には「起動後 2 秒間は腕を自然に下ろす」だけを案内する。

2.4.2.5 OrcSenderModule の出力フェーズ

`updateOutput()` は出力フェーズで毎周期実行される。`Udp.send()` は呼ばず、EMA の「モジュール間直接呼び禁止」に従い `SystemData` 経由で `OrcNetModule` に送信を依頼する。

1. `masterNow = millis()` を取得。指揮者は自分の `millis()` がそのままマスタ時刻。
2. **BEAT 送信予約 (イベント駆動)**: `data.beat.event` が立っていれば、`BeatPacket` を組み立てる。`header.seq` は `data.sender.beatSeq` をインクリメント。`header.timestampMs = masterNow`。`beatNo = data.beat.beatNo`。`playAtMasterMs = masterNow + beatLookaheadMs`。その上で `data.orcNet.pendingBeat` に格納、`pendingBeatRedundancy = beatRedundancy`、`hasPendingBeat = true` を立てる。実送信は同フェーズ後段の `OrcNetModule.updateOutput()` が担当。
3. **CTRL 送信予約 (周期駆動)**: 内部 `ctrlTimer` の経過 ms が `ctrlIntervalMs` (既定 50 ms = 20 Hz) 以上ならタイマをリセットし、`CtrlPacket` を組み立てる。`header.seq`

は `data.sender.ctrlSeq` をインクリメント, `header.timestampMs = masterNow`, `bpmQ8 = bpm × 8`, `velocity = data.tempo.velocity`, `state = data.conductor.state`, `data.orcNet.pendingCtrl` に格納し `hasPendingCtrl = true` を立てる. この `timestampMs` が楽器側の時計同期サンプルとして使われる.

2.4.3 楽器ノード (node_02-05)

4 台は同一コードで動かし, 差分は `ProjectConfig.h` と `score_data.h` (および `score_data.cpp`) にのみ閉じ込める.

2.4.3.1 モジュール構成

- ・ 入力配列: `OrcNetModule`, `OrcReceiverModule`
- ・ 出力配列: `NoteSenderModule`, `StatusLedModule`, `OrcNetModule`
- ・ ロジック: `applyPattern()` (時計同期更新・楽譜進行・細分音符予約・発音フラグ)

2.4.3.2 ProjectConfig (抜粋)

`node_02/include/ProjectConfig.h` の例. パート識別子は `PART_ID` (例: `0x02`) としてこのファイルで 1 か所だけ定義し, `ORC_RECEIVER_CONFIG.partId` と `NOTE_SENDER_CONFIG.partId` はともにこれを参照する (具体値をモジュール本体や複数の `Config` に重複して書かない). `node_03-05` は `PART_ID` と `startBeatNo` (および楽譜配列) のみ差し替える.

ORC_NET_CONFIG (楽器)

キー	値	役割
<code>mode</code>	<code>Sta</code>	<code>node_01</code> SoftAP に接続
<code>ssid</code>	<code>"OrchestraAP"</code>	接続先 SSID (指揮者と同一)
<code>pass</code>	<code>"orchestra2026"</code>	WPA2-PSK
<code>multicastIp</code>	<code>239.0.0.1</code>	指揮者と同一グループ
<code>udpPort</code>	<code>5001</code>	共通ポート

ORC_RECEIVER_CONFIG

キー	値	役割
partId	PART_ID	パート識別子 (PART_ID = 0x02, node_02 = 金管 1, 他ノードは 0x03-0x05)
startBeatNo	0	このパートの入り拍. beatNo がこの値に達したら楽譜の先頭 (kScore[0]) から鳴らし始める (localBeat = beatNo - startBeatNo が負の間は入り待ち). 主旋律 1 番 (node_02) は 0, 2 番・3 番は曲頭から数拍ずらす (具体値は楽譜確定時に決める)
clockSyncEmaAlpha	0.10	時計オフセット EMA 係数
clockSyncMinSamples	5	時計同期の収束判定の目安. Playing 遷移条件には使わない (収束待ちで遷移をブロックすると音が出なくなりやすいため). 診断ログとデバッグ表示にのみ使用
loopIntervalMs	5	ループ周期. main.cpp 側で間引きして UDP 受信遅延を抑える

NOTE_SENDER_CONFIG

キー	値	役割
baudRate	115200	自パート Mac の USB CDC ボーレート (CDC では設定値は実質無視されるが両端で揃える)
partId	PART_ID	NOTE ペイロード内識別子 (ORC_RECEIVER_CONFIG と同じ PART_ID を参照)

STATUS_LED_CONFIG

キー	値	役割
pin	LED_BUILTIN	表示ピン
blinkIntervalMs	500	既定点滅周期

2.4.3.3 SystemData (抜粋)

楽器ノード固有のロジック中間状態として SyncLogicData (時計同期) と ReceiverLogicData (保留 BEAT キュー) を集約する.

PerformerState (列挙, 3 状態)

値	意味
Idle	起動直後. SoftAP 未接続
WaitStart	SoftAP 接続済, 最初の BEAT 待ち
Playing	通常演奏. 受信 BEAT の beatNo に追従して楽譜位置を算出し発火する. BEAT が長く来なくても Playing に留まり次 BEAT で再開

SyncLogicData (時計同期)

フィールド	役割
offsetMs	自時計 → マスタ時刻の補正. $\text{masterNow} = \text{millis}() + \text{offsetMs}$
sampleCount	EMA に積んだサンプル数. clockSyncMinSamples 以上で「収束」と表示する (状態遷移条件には使わない)
converged	収束判定フラグ

PendingBeat (マスタ時刻待ち BEAT)

フィールド	役割
valid	有効エントリか
beatNo	拍番号 (重複排除キー)
playAtMasterMs	発音指定マスタ時刻
enqueuedAtMs	受信時の自時計時刻. デバッグ / 遅延計測用

ReceiverLogicData (受信ロジック中間状態)

フィールド	役割
hasFirstBeat	起動以降に最初の BEAT を受理したか. WaitStart → Playing 遷移条件
lastBeatNo	直近に受理した拍番号. これと同じ beatNo は重複排除
lastBeatMs	直近に受理した自時計時刻. 診断ログ用
pending	マスタ時刻待ち BEAT (PendingBeat). 発火後は valid = false で消化

ScoreProgressData (楽譜進行)

フィールド	役割
lastFiredIndex	直近に発火した kScore[] のインデックス (診断・ログ用). 発火位置そのものは状態として持たず, BEAT 発火のたびに受信 beatNo から $idx = (beatNo - startBeatNo) \bmod kScoreLength$ で算出する
pendingSub	細分音符 (8 分音符等) の予約スロット有効フラグ
pendingSubAtMs	細分音符を発火する自時計時刻 (拍頭 + subOffsetQ8 換算)
pendingSubNote / Velocity / DurationMs	細分音符の発音情報. ScoreEvent.subNote に応じて fireScoreEvent が積む

2.4.3.4 楽譜データ形式

楽譜は C 配列としてヘッダに埋め込み, SD カード等を使わない. score_data.h に ScoreEvent の宣言と楽譜本体 (kScore[]) の前方宣言を置き, 本体は同名の score_data.cpp に並べる.

ScoreEvent (楽譜 1 イベント)

フィールド	役割
beatAt	このイベントが何拍目かを示す参考値 (kScore[] は曲頭から 1 拍ずつ休符込みで並べ kScore[i].beatAt == i+1 となる). 発火位置は受信 beatNo から $\text{idx} = (\text{beatNo} - \text{startBeatNo}) \bmod \text{kScoreLength}$ で算出するため発火条件には使わず, 楽譜行を読みやすくするための冗長情報である
noteNumber	MIDI ノート番号. 0 なら休符
velocity	個別 velocity. CTRL の velocity と乗算合成
durationQ8	拍の 1/256 単位の発音長 (256 = 1 拍). Processing 側はこの値を ms に変換した durationMs で自動消音
flags	bit0 = NoteOn / bit1 = NoteOff (拡張用) / bit2 = 休符
subNote	細分音符 (拍頭から subOffsetQ8 拍ずれて鳴る 2 音目). 0 なら細分音符なし. 8 分音符・付点 8 分などの表現に使う
subVelocity	細分音符の velocity
subOffsetQ8	拍頭からのオフセット (128 = 半拍 = 8 分音符)
subDurationQ8	細分音符の発音長

	kScore[]	ScoreEvent の配列. 曲頭から 1 拍ずつ (休符込み) 時系列順に並べる. 発火位置の算出で beatNo を kScoreLength で剰余するため, 末尾の次は自動的に曲頭に戻ってループ再生される
楽譜配列の公開シンボル	kScoreLength	kScore[] の要素数 (曲の総拍数)

NoteOff の扱い: node_02-05 は NoteOff パケットを送らず, **Mac 側 Processing が NotePacket.durationMs を読み取って自動消音** する. Arduino 側から NoteOff を送る方式は USB CDC のバッファリング・順序待ちで「ノートが鳴りっぱなし」になりやすいため採らない. flags bit1 (NoteOff) は将来用の予約で, 現設計では使わない.

2.4.3.5 applyPattern() の処理フロー

1. **予約細分音符の発火判定:** score.pendingSub が立っており $\text{now} \geq \text{pendingSubAtMs}$ なら noteOut.pendingOn を立てて pendingSub をクリア.
2. **時計オフセット更新:** 受信フェーズで data.orcNet.hasNewCtrl OR hasNewBeat が立っていれば, $\text{sample} = \text{pkt.header.timestampMs} - \text{millis}()$ を EMA (clockSyncEmaAlpha)

で `data.sync.offsetMs` に積む。収束判定 (`sampleCount ≥ clockSyncMinSamples`) はデバッグ表示用のフラグに留め、状態遷移条件には使わない。

3. **演奏状態遷移 (3 状態)** : `Idle` → `WaitStart` (WiFi 接続完了) → `Playing` (最初の BEAT を受信, `receiver.hasFirstBeat` が立った)。仮想 BEAT を内挿して走り続ける機能は持たない。
4. **マスタ時刻発音判定**: `state == Playing` かつ `receiver.pending.valid` なら, `targetLocalMs = pending.playAtMasterMs - sync.offsetMs` を計算。 `waitMs = targetLocalMs - now` が
 - ・ ≤ 0 (既に到来 or 受信遅延) : **即発火**。次の step 5 に `pending.beatNo` を渡し, `pending.valid = false` で消化。受信遅延が大きくても捨てない (捨てるとうるさい症状の原因になるため)。
 - ・ > 0 : 何もしない。次ループで再評価し時刻到来したら発火。
5. **楽譜進行 (beatNo 駆動)** : step 4 で発火した BEAT があれば, その `beatNo` から `localBeat = beatNo - startBeatNo` を計算する。 `localBeat < 0` (まだこのパートの入り拍に達していない) なら何も鳴らさない (輪唱の入り待ち)。そうでなければ `idx = localBeat mod kScoreLength` で `kScore[idx]` を発火し, `score.lastFiredIndex = idx` を記録する (剰余により曲頭に戻ってループ再生される)。 `kScore[idx].subNote != 0` なら `score.pendingSub` に細分音符の予約を積む (次回以降のループで step 1 が消化)。BEAT を 1 個取りこぼしてもその次の BEAT の `beatNo` は欠落ぶんだけ進んでいるため, 算出される `idx` も正しい位置に飛び自己修復する。
6. **velocity 合成**: 楽譜 `velocity` と CTRL `velocity` を $\lfloor (\text{score} \times \text{ctrl}) / 127 \rfloor$ で乗算合成。ストレッチ未実装時は CTRL `velocity` が固定 64。
7. **LED 反映**: `performer.state` に応じて led 出力を更新。

1 ループ 1 NoteOn の制約 (既知の制約) : `noteOut` は 1 スロットしか持たないため, 細分音符 (step 1) と 4 分音符 (step 5) が偶然同一ループに重なると後勝ちで上書きされる。通常の BPM・受信ジッタの範囲では発生しないが, 「`noteOut` を 2 スロット化」または「同一ループに重なったら細分音符を次ループへ繰り越す」のいずれで安全側に倒すかは実装後の結合確認で決める (§2.5.3)。

`loop()` は `loopIntervalMs` (5 ms) で間引き, マスタ時刻判定の粒度を楽器間で揃える。

NOTE 送出は出力フェーズで `NoteSenderModule` が担当する。 `data.noteOut.pendingOn` を見て 20 バイトの `NotePacket` (`gate = 1`, `durationMs`) を組み立て, `Serial.write()` の直後に `Serial.flush()` を呼んで USB CDC の内部バッファを即吐く (バッファに溜まると発音がバースト化するため)。 `NoteOff` は送らず, Processing 側が `durationMs` 経過時点で自動消音する。

2.4.3.6 ノード間差分

4 台の楽器は ProjectConfig.h と score_data.h 以外は同一コードで、差分は PART_ID ・ startBeatNo ・ 楽譜配列のみ（表 2.5）。輪唱は startBeatNo を差し替えるだけで、主旋律 2 番・3 番が曲頭から数拍遅れて入る。

表 2.5: 楽器ノード間差分（startBeatNo のずらし拍数は
楽譜確定時に決める。以下は例）

ノード	PART_ID	startBeatNo	score_data
node_02	0x02 (金管 1)	0	主旋律 1 番（先頭から）
node_03	0x03 (金管 2)	4	主旋律 2 番（輪唱で 4 拍遅れて入る）
node_04	0x04 (金管 3)	8	主旋律 3 番（輪唱でさらに遅れて入る）
node_05	0x05 (ドラム)	0	リズム伴奏（先頭から）

beatAt は楽譜行に拍番号（1, 2, 3, ...）を併記して読みやすくするための参考値で、発火位置は受信 beatNo と startBeatNo から算出するため発火条件には使わない。

2.5 テストの設計

本節では、本システムの設計が満たすべき数値目標（設計目標値）、それを段階的に確認していくための検証の方向性、および実機検証で詰める仮値・未確定事項を示す。具体的な試験ケースや計測手順は実装フェーズで確定する。

2.5.1 設計目標値

本システムの設計が狙う数値目標を表 2.6 に示す。これらは「テストの可否基準」としてではなく、設計判断の前提として置いた値であり、実機計測で再調整する想定である。

表 2.6: 設計目標値

項目	目標値	根拠・位置づけ
楽器間 同期誤差	$\leq 20 \text{ ms}$	ADR-0006 でチーム合意した有効性指標。合奏では数十 ms 程度のタイミングのズレは演奏者・聴き手ともほとんど気づかれないという知見を踏まえ、安全側に 20ms と置いた。 §2.3.3.6 のマスタクロック方式はこの水準を狙う
拍検出 正解率	$\geq 90 \%$	定速指揮時に演奏破綻を起こさない実用水準として置いた値。動加速度ノルム閾値による振り下ろし検出 (§2.4.2) で達成を見込む
テンポ追従 遅延	$\leq 2 \text{ 拍}$	人間の指揮者・演奏者が BPM 変化を 1-2 拍で追従するのと同等を最低水準とした。BPM 推定の EMA 係数 (bpmEmaAlpha) でこの範囲に収める
入力フェーズ CPU 時間	$\leq 2 \text{ ms}$	IMU を 200 Hz (周期 5 ms) で読むため、3 フェーズ (入力 → applyPattern() → 出力) の各フェーズが $5/3 \approx 1.7 \text{ ms}$ 以内に収まる必要がある。入力に 2ms 以下の制約を置き、残りをロジックと出力に割り当てる
起動時間	$\leq 5 \text{ s}$	SoftAP 起動・STA 接続・初期 DHCP で 2-3 秒、指揮者の Calibrating (停止時ノルムの学習) に 2 秒を見込み、合計 5 秒以内に最初の CTRL を送出する想定
強弱制御 (スト レッチ)	3 段階以上 (p / mf / f)	西洋音楽の強弱記号の基本単位が p / mf / f の 3 段階であり、初級～中級曲のダイナミクスを最低限再現できる。ストレッチ機能 F1.5 の達成水準

2.5.2 検証の方向性

設計の妥当性は、実装後に「単体 → 結合 → 実機」の 3 段階で確認していく方針とする。

- ・ **単体**: EMA に従い判断ロジックを IModule 派生ではなく applyPattern() 関数に書くため、SystemData をテスト側で組み立てて applyPattern() を呼ぶだけで、拍検出・テン

ポ推定・楽譜進行・時計同期・発音判定・細分音符予約のロジックを切り出して確認できる設計とする。OrcProtocol のシリアライズ／デシリアライズもバイト列のラウンドトリップで確認できる。

- ・ **結合**: 指揮者ノード単体 → 指揮者 + 楽器 1 台 → 指揮者 + 楽器 4 台, と接続規模を段階的に広げ, BEAT 受信から NOTE 出力までの経路と, 4 台の発音タイミング差 (楽器間同期誤差) を確認していく。
- ・ **実機 (システム)**: 全ノード + PC (Processing) で実音を再生し, 指揮の速度・振幅を変えたときのテンポ追従・強弱の挙動を聴感と計測の両面で確認する。受信 BEAT を意図的に一部破棄した条件での演奏継続もこの段階で確認する。

具体的な試験ケース・計測方法 (サンプル数, ログ取得手順, 合否の判定式など) は実装フェーズで詰める。

2.5.3 実機検証で確定する仮値・未確定事項

設計値のうち実機での段階的な結合確認で詰めるもの, および楽譜・音色の仕様確定を待って実装するものを表 2.7 に列挙する。

表 2.7: 実機で確定する仮値・未確定事項

項目	仮値・現状	確定の見極め
BEAT 連発回数	beatRedundancy = 2	実装後の結合確認でパケロス・同期誤差を見て 1-3 連発で調整
lookahead	beatLookaheadMs = 50	実装後の結合確認で, 振り→音の体感遅延と, 最も遠い楽器が waitMs ≤ 0 になる比率を見て 30-80 ms で調整 (STA の WiFi 省電力 OFF が効かなければ 100-150 ms)
時計同期 EMA 係数	clockSyncEmaAlpha = 0.10	実装後の結合確認で offsetMs の収束時間と定常ジッタを見て 0.05-0.20 で調整
片道遅延の系統差	4 台でほぼ同程度と仮定し補正なし	実装後の結合確認で 4 台の同期誤差を見て, 系統差が無視できなければ往復測定で各台の片道遅延を較正し offsetMs に固定補正值として足し込む (§2.3.3.6)

項目	仮値・現状	確定の見極め
WiFi STA 省電力モード	無効化して運用 (WiFiS3 での具体的な無効化手段は未確認)	実機で WiFiS3 の省電力 OFF API を確認. OFF にできなければ beatLookaheadMs を 100-150ms に拡大, もしくは指揮者側を CTRL/BEAT のユニキャスト連送に切替 (§2.2)
1 ループ 1 NoteOn の制約	noteOut は 1 スロット (細分音符と 4 分音符が同一ループに重なると後勝ち上書き)	実装後の結合確認で発生頻度を見て, 「noteOut の 2 スロット化」か「同一ループに重なったら細分音符を次ループへ繰り越し」のどちらかに決める (§2.4.3)
WiFi チャンネル	channel = 6	会場の電波状況を Site Survey して空きチャンネルへ
LED 色・点滅周期	1/2/5 Hz の点滅	視認性確認後に決定
強弱推定 (ストレッチ)	振り幅 → velocity 線形写像の 3 段階 (現状は CTRL.velocity = 64 固定でスタブ)	実機確認で p / mf / f が分離して聞こえる写像式に調整. 必達機能の完成後に着手
ビブラート (ストレッチ)	角速度の高周波成分から振幅変調 (仮)	必達機能の完成後に着手. Mac 側でフィルタ / LFO を適用
ドラム node_05	金管同様の ScoreEvent で実装 (仮)	主旋律 3 パートの完成後に着手. noteNumber は GM ドラムマップ (36=バスドラム, 38=スネア, 42=クロースドハイハット 等) に固定する
楽譜データスキーマ	ScoreEvent (本書の定義)	楽譜エディタ (齋藤担当) の {pitch, durationBeats} 形式 → ScoreEvent 配列を生成する変換層で吸収. 変換層は楽器ノード側 (塩澤) が score_data.cpp 生成スクリプトとして用意する想定で, 具体形式は楽譜エディタの出力仕様が固まり次第確定する
ESP-NOW (ストレッチ)	不採用 (WiFi UDP マルチキャストを主案とする)	必要が出れば再検討するストレッチ拡張候補として据え置き

第 3 章 生成 AI の利用について

本書の執筆にあたっては、生成 AI として **Claude Opus (Anthropic)** を補助的に利用した。提出規約上、本計画書については生成 AI 利用申告書の提出が明示的に求められているわけではないが、透明性のため、どのように利用したかを簡潔に記しておく。

利用方法は **対話を通じた共同執筆** である。設計の方針、章立て、図表の構成、文章表現の選び方などについて Claude Opus と繰り返し議論し、提案された文案を著者が精査・修正・取舍選択しながら本書の形にまとめた。コード例や設計値（ピン配置・通信パラメータ・状態遷移など）は、著者が自ら設計・検証したものを正としており、AI による出力をそのまま採用した箇所はない。

AI の出力に対しては、著者の Embedded-Module-Architecture (EMA) の仕様および Arduino/ESP32 の実装制約と整合するかを著者自身がレビューし、事実誤認や設計と食い違う記述は都度書き換えた。本書に残る記述の妥当性および誤りの責任は、すべて著者が負う。